

Git + GitHub Interview — Lecture 1

Q1. What is Git?

Git is a **distributed version control system** used to track changes in code. It helps developers manage versions and collaborate on projects.

Interviewer may ask: *Why do we use Git?*

Q2. What is GitHub?

GitHub is a **cloud-based platform** used to store Git repositories online. It helps developers collaborate using features like Pull Requests, Issues and Actions.

May ask: *Git and GitHub are same?*

Q3. Git vs GitHub?

Git is the version control tool installed on your computer.

GitHub is an online platform where Git repositories can be hosted and collaborated on.

Q4. What is a Git repository?

A repository is a folder where Git **tracks your project's files and their history**. It contains the project files along with Git's version-control information.

May ask: *What is a repo?*

Q5. What is git init?

git init initializes Git in a project folder.

It creates a hidden .git folder that stores Git's tracking information.

Q6. What is `git clone`?

`git clone` creates a **local copy of a remote repository** on your computer. It also connects your local repository with the remote repository.

Q7. What is the difference between `git add`, `git commit`, and `git push`?

`git add` moves changes to the **staging area**.
`git commit` saves them locally, while `git push` uploads commits to the remote repository.

Q8. What is the staging area?

The staging area contains changes that are **selected for the next commit**. We use `git add` to move changes into staging.

Q9. What is a Git commit?

A commit is a **snapshot of changes** saved in Git's history. It allows us to track what changed and when.

Q10. What is `git status`?

`git status` shows the current state of your working directory. It tells you about modified, staged, and untracked files.

Q11. What is `git log`?

`git log` displays the **commit history** of a repository. It shows information like commit ID, author, date and message.

Q12. What is a Git branch?

A branch is an **independent line of development** in Git.
It lets us work on features without directly affecting the main branch.

Q13. Why do we use branches?

Branches allow developers to work on different features or fixes separately.
After completion, the changes can be merged into the main branch.

Q14. What is git merge?

git merge combines changes from one branch into another branch.
For example, we can merge a feature branch into main.

Q15. What is a merge conflict?

A merge conflict occurs when Git cannot automatically combine changes.
Usually, it happens when the same part of a file was changed differently in two branches.

Q16. How do you resolve a merge conflict?

First, open the conflicted file and decide which changes to keep.
Then remove conflict markers, run git add, and create a commit.

Q17. What is git pull?

git pull **downloads changes from the remote repository and merges them** into the current branch.
It is basically a combination of fetch and merge.

Q18. git pull VS git fetch?

git fetch downloads remote changes but **does not modify your current branch**.
git pull downloads the changes and also integrates them into your branch.

Q19. What is git push?

git push uploads your **local commits to a remote repository** such as GitHub.
It is commonly used after committing your changes.

Q20. What is origin in Git?

origin is the default name given to the **remote repository** when we clone or add one.

For example, git push origin main pushes the main branch to that remote.

🔥 **Next part:** rebase, stash, reset, revert, cherry-pick, .gitignore, Pull Request, Fork + important real-world scenarios.

next

Git + GitHub — Lecture 1 (Part 2) 🔥

Q21. What is git rebase?

git rebase moves your branch's commits on top of another branch.
It creates a **cleaner linear history** compared to merge.

May ask: *Merge vs Rebase?*

Q22. git merge VS git rebase?

Merge combines branches and usually creates a merge commit.
Rebase moves commits and keeps the history more linear.

Q23. What is git stash?

git stash temporarily saves your **uncommitted changes**.

It is useful when you need to switch branches without committing unfinished work.

Q24. How do you get stashed changes back?

Use git stash pop to restore the latest stashed changes and remove it from stash.

git stash apply restores it but keeps the stash.

Q25. What is git reset?

git reset moves the current branch to another commit and can modify staging/working files.

It is commonly used to **undo local commits or changes**.

Q26. git reset VS git revert?

reset can remove commits from the branch history, so it is mainly used for local changes.

revert creates a **new commit that undoes** an earlier commit and is safer for shared branches.

Q27. What is git cherry-pick?

git cherry-pick applies a **specific commit** from another branch to your current branch.

It is useful when you need only one particular change.

Q28. What is .gitignore?

.gitignore specifies files and folders that Git should **not track**.
For example: .env, node_modules, logs and build files.

Q29. Why should .env not be pushed to GitHub?

.env may contain **API keys, database passwords or other secrets**.
These should remain private and be stored through environment variables/secrets.

Q30. What if you accidentally push a secret to GitHub?

First, **revoke/rotate the exposed secret immediately**.
Then remove it from Git tracking/history and push the corrected changes.

Q31. What is a Pull Request (PR)?

A Pull Request is a request to **merge changes from one branch into another**.
It allows teammates to review and discuss code before merging.

Q32. What is a fork in GitHub?

A fork creates your **own copy of someone else's GitHub repository** under your account.
You can modify it without directly changing the original repository.

Q33. Fork vs Clone?

Fork creates a repository copy on your GitHub account.
Clone downloads a repository from GitHub to your local computer.

Q34. What is git remote?

git remote manages connections between your local repository and remote repositories.

For example, git remote -v shows the remote URLs.

Q35. What is git diff?

git diff shows the **changes made in files** that have not been committed.

It helps you review your changes before committing.

Q36. What is GitHub Actions?

GitHub Actions is a **CI/CD automation tool** built into GitHub.

It can automatically run tests, build projects and deploy applications.

Q37. What is a Personal Access Token (PAT)?

A Personal Access Token is used to **authenticate Git operations with GitHub**.

It can be used instead of a password for HTTPS authentication.

Q38. Scenario: You committed the wrong file. What will you do?

If it is not pushed, I can undo the commit using git reset and correct the files.

Then I will stage the correct files and commit again.

Q39. Scenario: Your teammate changed the same code and now you have a conflict. What will you do?

I will pull/fetch the latest changes and identify the conflicted files.

Then I will resolve the conflict, test the code, and commit the resolved changes.

Q40. Scenario: You pushed code but GitHub says the push was rejected. What will you check?

I will first check the **exact error message**, branch status and remote configuration.

Then I will fix the reported issue, such as a conflict, secret, or permission problem, and push again.

✅ **Git + GitHub theory/interview lecture complete: 40 questions.**

These are enough for a **fresher/service-based MERN interview**; no need to memorize 100+ Git questions.

superb now mongodb jaisa ki tumne upar bola tha vo sb bhi dedo mongoose ke bhi

Bilkul 🤓 **MongoDB + Mongoose ka complete interview lecture** karte hain, exactly Git/Node wale format mein.

Target: ~60 theory/interview questions + important scenario-based questions

Answer: maximum **3 lines**

No faltu / no repeated questions

MongoDB aur Mongoose dono cover honge — jo fresher MERN interview mein genuinely important hain.

MongoDB + Mongoose — Lecture 1 🚀

Q1. What is MongoDB?

MongoDB is a **NoSQL, document-based database** that stores data in BSON documents.

It is commonly used in MERN applications.

May ask: *What type of database is MongoDB?*

Q2. Why do we use MongoDB?

MongoDB provides a flexible document structure and works well with JavaScript applications.

It is useful when application data can change or grow easily.

Q3. What is a document in MongoDB?

A document is a **single record** stored in MongoDB in BSON format.

It is similar to a row in a relational database.

Q4. What is a collection?

A collection is a group of MongoDB documents.

It is roughly equivalent to a **table in SQL**.

Q5. MongoDB vs SQL database?

MongoDB stores data as **documents and collections**, while SQL databases use rows and tables.

MongoDB is schema-flexible, whereas SQL databases generally use a predefined schema.

Q6. What is BSON?

BSON stands for **Binary JSON**.

MongoDB uses BSON because it supports additional data types and is efficient for database operations.

Q7. What is _id in MongoDB?

_id is the **unique identifier** of each document.

MongoDB automatically creates it if we don't provide one.

Q8. What is ObjectId?

ObjectId is MongoDB's commonly used type for the `_id` field. It generates a practically unique identifier for documents.

Q9. What does schema-less mean in MongoDB?

MongoDB does not require every document in a collection to have exactly the same structure. This gives applications more flexibility.

Q10. What is Mongoose?

Mongoose is an **ODM (Object Data Modeling) library for MongoDB and Node.js**. It provides schemas, models, validation and convenient database operations.

Q11. MongoDB vs Mongoose?

MongoDB is the **actual database** that stores the data. Mongoose is a Node.js library that helps us interact with MongoDB using schemas and models.

Q12. What is a Mongoose Schema?

A Schema defines the **structure, fields, data types and rules** for documents. For example, we can define name: String and age: Number.

Q13. What is a Mongoose Model?

A Model is created from a Schema and is used to **interact with a MongoDB collection**. We use methods like `find()`, `create()` and `updateOne()` through models.

Q14. Schema vs Model?

Schema defines how the data should look.

Model uses that schema to perform operations on the MongoDB collection.

Q15. What is CRUD?

CRUD stands for **Create, Read, Update and Delete**.

These are the four basic operations performed on database data.

Q16. How do you insert a document?

We can use MongoDB methods like `insertOne()` or Mongoose methods like `Model.create()`.

They add new documents to a collection.

Q17. What is `find()`?

`find()` returns **multiple documents** matching a condition.

In Mongoose, it returns a query that can be executed to get the results.

Q18. `find()` VS `findOne()`?

`find()` can return multiple matching documents.

`findOne()` returns only the **first matching document**.

Q19. What is `findById()` in Mongoose?

`findById()` finds a document using its `_id`.

It is commonly used when fetching a specific resource by ID.

Q20. What is `findByIdAndUpdate()`?

It finds a document using its `_id` and updates it.

It is commonly used in APIs such as updating a user's profile.

Q21. What is an index in MongoDB?

An index improves the **speed of queries** on specific fields.

Without a suitable index, MongoDB may need to scan many documents.

Q22. What is a unique index?

A unique index prevents duplicate values in a field.

For example, we can make the email field unique.

Q23. What happens if we create too many indexes?

Indexes improve reads but consume **storage and memory**.

They can also slow down insert and update operations because indexes must be maintained.

Q24. What is aggregation?

Aggregation is used to **process and transform documents** and produce calculated results.

It is useful for reports, statistics and complex data processing.

Q25. What is an aggregation pipeline?

An aggregation pipeline processes documents through multiple stages.

Common stages include `$match`, `$group`, `$sort`, `$project` and `$lookup`.

Q26. What is \$match?

\$match filters documents based on a condition.
It is similar to the WHERE clause in SQL.

Q27. What is \$group?

\$group groups documents based on a specified field or expression.
It is commonly used with operators like \$sum, \$avg, \$min and \$max.

Q28. What is \$lookup?

\$lookup is used to **combine data from another collection**.
It is similar to a JOIN operation in SQL.

Q29. What is populate() in Mongoose?

populate() replaces referenced IDs with the **actual related documents**.
It is commonly used when using references between collections.

Q30. Embedding vs Referencing?

Embedding stores related data inside the same document.
Referencing stores another document's ID and retrieves related data separately.

Q31. What is normalization and denormalization?

Normalization separates data to reduce duplication.
Denormalization duplicates some data to make reading data faster or simpler.

Q32. What is MongoDB validation?

Validation ensures that data follows defined rules before being stored.
Mongoose provides validations such as required, min, max, enum and match.

Q33. What is `required: true` in Mongoose?

It means the field must be provided when creating a document.
If it is missing, Mongoose validation fails.

Q34. What is `enum` in Mongoose?

`enum` restricts a field to a predefined set of values.
For example, `role` can be limited to `"user"` or `"admin"`.

Q35. What is a default value in Mongoose?

The `default` option automatically assigns a value when one isn't provided.
Example: `status: { type: String, default: "active" }`.

Q36. What is `lean()` in Mongoose?

`lean()` returns plain JavaScript objects instead of full Mongoose documents.
It can improve performance when we only need to read data.

Q37. What is `exec()` in Mongoose?

`exec()` executes a Mongoose query and returns a Promise.
It is commonly used with `await`.

Q38. What is `distinct()`?

`distinct()` returns unique values for a specified field.
For example, it can return all unique cities from user documents.

Q39. What is upsert?

Upsert means **update if the document exists, otherwise insert a new document**.

It is enabled using { upsert: true }.

Q40. What is countDocuments()?

countDocuments() returns the number of documents matching a condition.
It is commonly used for counts and pagination.

Q41. What is projection?

Projection controls **which fields are returned** from a query.
For example, we can return only name and email instead of the entire document.

Q42. What is \$project?

\$project is an aggregation stage used to **include, exclude or create fields** in the output.

It is similar to projection but works inside an aggregation pipeline.

Q43. What is \$unwind?

\$unwind breaks an array field into **separate documents**.

It is commonly used in aggregation when working with arrays.

Q44. What is \$push?

\$push adds a value to an array field.

It allows duplicate values.

Q45. \$push VS \$addToSet?

\$push adds values even if they already exist.

\$addToSet adds the value only if it is **not already present**.

Q46. What is \$pull?

\$pull removes elements from an array that match a specified condition.

It is useful for removing items from arrays.

Q47. What is \$inc?

\$inc increments or decrements a numeric field.

For example, { \$inc: { age: 1 } } increases age by 1.

Q48. What is \$unset?

\$unset removes a field from a document.

For example, { \$unset: { age: "" } } removes the age field.

Q49. What is a compound index?

A compound index is an index created on **multiple fields**.

It is useful when queries commonly filter or sort using those fields together.

Q50. What is a text index?

A text index allows MongoDB to perform **text searches** on string fields.

It is useful for searching words or phrases in text data.

Important Scenario Questions

Q51. Your MongoDB query is very slow. What will you check?

I would check the query conditions and whether appropriate **indexes** exist.
I would also use query analysis tools such as `explain()` to identify the problem.

Q52. Two users register with the same email. How will you prevent it?

I would create a **unique index** on the email field.
I would also handle the duplicate-key error properly in the API.

Q53. API is returning all user fields, but password should not be returned. What will you do?

I can exclude the password using projection, such as `.select("-password")`.
I can also define the field with `select: false` in the Mongoose schema.

Q54. You need user's bookings along with car details. What can you use?

If bookings contain a reference to the car, I can use **Mongoose populate()**.
For more complex processing across collections, MongoDB `$lookup` can be used.

Q55. A field should accept only user, admin, or manager. What will you use?

I would use Mongoose's **enum validation**.
This prevents invalid role values from being stored through Mongoose.

Q56. You need to update only one user's name. Which method?

I can use `findByIdAndUpdate()` if I have the user's ID.
For a condition other than ID, I can use `updateOne()`.

Q57. You need to update all users from one city. Which method?

I would use `updateMany()` with the city condition.
It updates all documents matching that condition.

Q58. You need to delete one specific user. Which method?

I can use `deleteOne()` with a condition.
If I have the ID, `findByIdAndDelete()` is also commonly used.

Q59. MongoDB connection fails when the server starts. What will you check?

I would first check the **MongoDB URI, environment variables and network/Atlas access**.
Then I would check the connection error message and database configuration.

Q60. Why shouldn't we store passwords directly in MongoDB?

Passwords should never be stored as plain text.
We should **hash passwords using a secure hashing algorithm such as bcrypt** before storing them.

✅ **MongoDB + Mongoose Lecture complete: 60 questions.**

Ye set **fresher MERN interview ke liye kaafi strong** hai. Iske baad MongoDB ke **query/coding questions alag lecture** mein karna best rahega—exactly jaise humne decide kiya tha.

wow yrr ha dedo query questions bhi important h jo bhi

Bilkul yrr 🔥 **MongoDB Query/Coding ka bhi complete important set** karte hain.

Sirf woh queries jo **MERN fresher interviews mein genuinely poochi ja sakti hain** — no random advanced stuff.

MongoDB Query Questions — Lecture

Assume collection: users

```
{
  name: "Rahul",
  age: 22,
  city: "Delhi",
  salary: 40000,
  skills: ["JavaScript", "React"],
  role: "user"
}
```

Q1. Insert one document

```
db.users.insertOne({
  name: "Rahul",
  age: 22,
  city: "Delhi"
})
```

Q2. Insert multiple documents

```
db.users.insertMany([
  { name: "Rahul", age: 22 },
  { name: "Priya", age: 24 }
])
```

Q3. Find all users

```
db.users.find()
```

Q4. Find users from Delhi

```
db.users.find({ city: "Delhi" })
```

Q5. Find one user

```
db.users.findOne({ name: "Rahul" })
```

Q6. Find users whose age is greater than 25

```
db.users.find({ age: { $gt: 25 } })
```

Q7. Find users whose age is less than 25

```
db.users.find({ age: { $lt: 25 } })
```

Q8. Find users whose age is between 20 and 30

```
db.users.find({  
  age: { $gte: 20, $lte: 30 }  
})
```

Q9. Find users whose city is Delhi OR Mumbai

```
db.users.find({  
  $or: [  
    { city: "Delhi" },  
    { city: "Mumbai" }  
  ]  
})
```

Q10. Find users who are from Delhi AND age > 25

```
db.users.find({  
  city: "Delhi",  
  age: { $gt: 25 }  
})
```

Q11. Find users whose role is either admin or manager

```
db.users.find({  
  role: { $in: ["admin", "manager"] }  
})
```

Q12. Find users whose role is NOT admin

```
db.users.find({  
  role: { $ne: "admin" }  
})
```

Q13. Find users whose city is NOT Delhi

```
db.users.find({  
  city: { $ne: "Delhi" }  
})
```

Q14. Find documents where email exists

```
db.users.find({  
  email: { $exists: true }  
})
```

Q15. Find users having JavaScript in skills

```
db.users.find({  
  skills: "JavaScript"  
})
```

Q16. Find users having both JavaScript and React

```
db.users.find({  
  skills: { $all: ["JavaScript", "React"] }  
})
```

Q17. Update one user's salary

```
db.users.updateOne(  
  { name: "Rahul" },  
  { $set: { salary: 50000 } }  
)
```

Q18. Update all users from Delhi

```
db.users.updateMany(  
  { city: "Delhi" },  
  { $set: { country: "India" } }  
)
```

Q19. Increase salary by 5000

```
db.users.updateOne(  
  { name: "Rahul" },  
  { $inc: { salary: 5000 } }  
)
```

Q20. Remove a field

```
db.users.updateOne(  
  { name: "Rahul" },  
  { $unset: { salary: "" } }  
)
```

Q21. Rename a field

```
db.users.updateMany(  
  {},  
  { $rename: { "name": "fullName" } }  
)
```

Q22. Add a skill to an array

```
db.users.updateOne(  
  { name: "Rahul" },  
  { $push: { skills: "Node.js" } }  
)
```

Q23. Add skill only if it doesn't already exist

```
db.users.updateOne(  
  { name: "Rahul" },  
  { $addToSet: { skills: "React" } }  
)
```

Q24. Remove a skill from array

```
db.users.updateOne(  
  { name: "Rahul" },  
  { $pull: { skills: "React" } }  
)
```

Q25. Delete one user

```
db.users.deleteOne(  
  { name: "Rahul" }  
)
```

Q26. Delete all users from Delhi

```
db.users.deleteMany(  
  { city: "Delhi" }  
)
```

Projection / Sorting / Pagination

Q27. Show only name and email

```
db.users.find(  
  {},  
  { name: 1, email: 1 }  
)
```

Q28. Hide password from result

```
db.users.find(  
  {},  
  { password: 0 }  
)
```

Q29. Sort users by salary descending

```
db.users.find().sort({ salary: -1 })
```

Q30. Sort users by age ascending

```
db.users.find().sort({ age: 1 })
```

Q31. Get top 5 highest-paid users

```
db.users.find()  
  .sort({ salary: -1 })  
  .limit(5)
```

Q32. Skip first 10 users

```
db.users.find().skip(10)
```

Q33. Pagination — page 2, 10 users per page

```
db.users.find()  
  .skip(10)  
  .limit(10)
```

Formula: (page - 1) * limit

Mongoose Queries — VERY IMPORTANT

Assume:

```
const User = mongoose.model("User", userSchema);
```

Q34. Create a user

```
const user = await User.create({  
  name: "Rahul",  
  age: 22  
});
```

Q35. Find all users

```
const users = await User.find();
```

Q36. Find users from Delhi

```
const users = await User.find({  
  city: "Delhi"  
});
```

Q37. Find one user by email

```
const user = await User.findOne({  
  email: "rahul@gmail.com"  
});
```

Q38. Find user by ID

```
const user = await User.findById(userId);
```

Q39. Update user by ID

```
const user = await User.findByIdAndUpdate(
  userId,
  { name: "Aman" },
  { new: true }
);
```

new: true returns the **updated document**.

Q40. Delete user by ID

```
await User.findByIdAndDelete(userId);
```

Q41. Select only name and email

```
const users = await User.find()
  .select("name email");
```

Q42. Exclude password

```
const users = await User.find()
  .select("-password");
```

Q43. Sort using Mongoose

```
const users = await User.find()
  .sort({ salary: -1 });
```

Q44. Limit results

```
const users = await User.find()
  .limit(10);
```

Q45. Count users

```
const count = await User.countDocuments();
```

Aggregation — MUST KNOW

Q46. Find all users from Delhi using aggregation

```
db.users.aggregate([
  { $match: { city: "Delhi" } }
])
```

Q47. Calculate average salary

```
db.users.aggregate([
  {
    $group: {
      _id: null,
      avgSalary: { $avg: "$salary" }
    }
  }
])
```

Q48. Find total salary

```
db.users.aggregate([
  {
    $group: {
      _id: null,
      totalSalary: { $sum: "$salary" }
    }
  }
])
```

Q49. Count users by city

```
db.users.aggregate([
  {
    $group: {
      _id: "$city",
      totalUsers: { $sum: 1 }
    }
  }
])
```

Q50. Find highest salary

```
db.users.aggregate([
  {
    $group: {
      _id: null,
      maxSalary: { $max: "$salary" }
    }
  }
])
```

Q51. Sort aggregation result

```
db.users.aggregate([
  { $sort: { salary: -1 } }
])
```

Q52. Use \$project

```
db.users.aggregate([
  {
    $project: {
      name: 1,
      salary: 1
    }
  }
])
```

Q53. Find users with salary > 50000 and sort them

```
db.users.aggregate([
  { $match: { salary: { $gt: 50000 } } },
  { $sort: { salary: -1 } }
])
```

Q54. Count users

```
db.users.aggregate([
  {
    $count: "totalUsers"
  }
])
```

Q55. Find unique cities

```
db.users.distinct("city")
```

Mongoose Relationships

Q56. Populate user's booking details

```
const bookings = await Booking.find()
  .populate("user");
```

Q57. Populate multiple fields

```
const bookings = await Booking.find()
  .populate("user")
  .populate("car");
```

Q58. Populate only selected fields

```
const bookings = await Booking.find()
  .populate("user", "name email");
```

Interview Scenario Queries

Q59. Find the second-highest salary

```
db.users.find()  
  .sort({ salary: -1 })  
  .skip(1)  
  .limit(1)
```

Q60. Find users whose salary is between 30k–50k

```
db.users.find(  
  salary: { $gte: 30000, $lte: 50000 }  
)
```

Q61. Find users who have more than 2 skills

```
db.users.find(  
  skills: { $size: 3 }  
)
```

⚠ \$size: 3 means **exactly 3**, not more than 2.

Q62. Find users whose name starts with “A”

```
db.users.find(  
  name: { $regex: /^A/i }  
)
```

Q63. Find users whose name contains “an”

```
db.users.find(  
  name: { $regex: /an/i }  
)
```

Q64. Update salary for all users by 10%

```
db.users.updateMany(  
  {},  
  [  
    {  
      $set: {  
        salary: { $multiply: ["$salary", 1.1] }  
      }  
    }  
  ]  
)
```

Q65. Find duplicate emails

```
db.users.aggregate([  
  {  
    $group: {  
      _id: "$email",  
      count: { $sum: 1 }  
    }  
  },  
  {  
    $match: {  
      count: { $gt: 1 }  
    }  
  }  
])
```

★ Sabse important jo pakka practice karna

Q4, Q8, Q9, Q10, Q11, Q17–24, Q27–33, Q36–43, Q47–53, Q55, Q57, Q59, Q62, Q65

Ye **MongoDB + Mongoose query part** interview ke liye strong kar dega.

Iske baad MongoDB ke queries ko aur badhane ki zarurat nahi—**inhe khud se likhne ki practice karna zyada important hai.**

ok ab tum js , react, node and express mpngodb ke jo practical important question h jo mujhe live kra skte h sb dedo jaise hota h na phele definition bta dete h then jaake boldete h ki bnake dikhao to vhi sb now start important dena codinng queestion jo puche jaate h bus interviewer kya puch skta h vo question